

Mise en place de CI/CD avec Jenkins pour un projet Node.js

Table des Matières

1. Introduction
2. Installation de jenkins
3. Configuration du Projet Node.js
4. Création du Pipeline CI/CD
 - Build
 - Test
 - Déploiement sur docker
5. Deploy to minikube

1. Introduction

Cette documentation explique comment mettre en place un pipeline CI/CD avec jenkins pour un projet Node.js. L'objectif est d'automatiser le processus de build, test, release, et déploiement de l'application.

2. Installation de jenkins avec docker

Prérequis:

- VM ubuntu 22.04 LTS sur Virtual Box
- Docker
- jre11et jdk11 :

`sudo apt install default-jre`

`sudo apt install default-jdk`

1. installation sans docker

- `wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key |sudo gpg --dearmor -o /usr/share/keyrings/jenkins.gpg`

- `sudo sh -c 'echo deb [signed-by=/usr/share/keyrings/jenkins.gpg] http://pkg.jenkins.io/debian-stable binary/ > /etc/apt/sources.list.d/jenkins.list'`
- `sudo apt update`
- `sudo apt install Jenkins`
- `sudo systemctl status Jenkins`

2. installation avec docker

Docker Compose pour jenkins:

- Créez un dossier nommé «jenkins »
- Créez un fichier `docker-compose.yml` avec le contenu suivant:

```
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - ./jenkins:/var/jenkins
    restart: always

volumes:
  jenkins:
```

Démarrer le service :

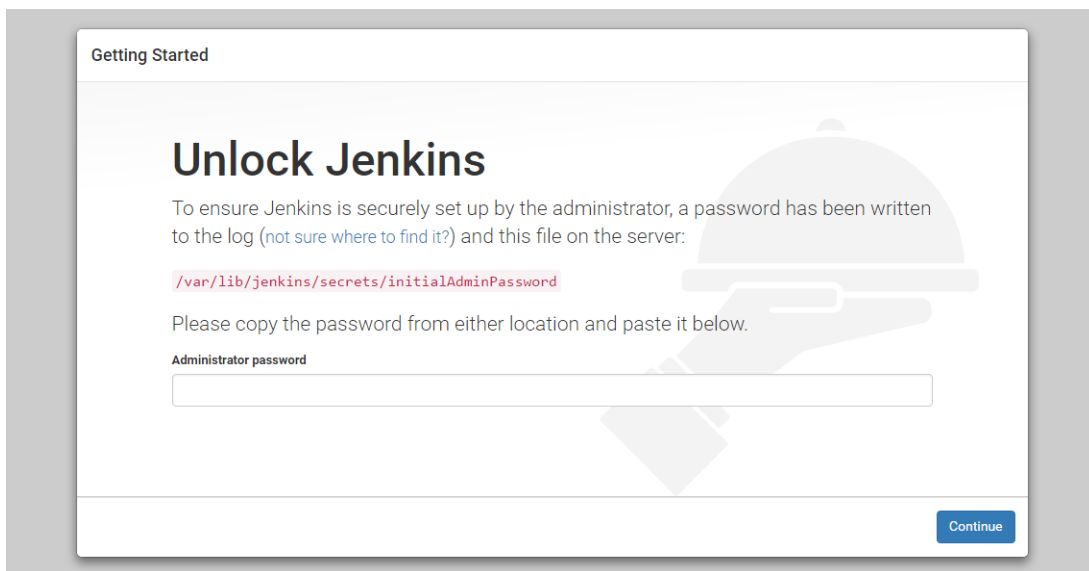
`docker-compose up -d`

L'interface web de jenkins

Configuration de Jenkins

Pour configurer votre installation, consultez Jenkins sur son port par défaut, 8080 en utilisant votre nom de domaine ou l'adresse IP de votre serveur : `http://your_server_ip_or_domain:8080`

Vous devriez voir apparaître l'écran **Unlock Jenkins** qui affichera l'emplacement du mot de passe initial :



Dans la fenêtre du terminal, utilisez la commande `cat` pour afficher le mot de passe :

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Copiez le mot de passe alphanumérique composé de 32 caractères du terminal et collez-le dans le champ **Administrator password**, puis cliquez sur **Continue**.

L'écran suivant présente l'option d'installer des plugins suggérés ou de sélectionner des plugins spécifiques :

Customize Jenkins

Plugins extend Jenkins with additional features to support many different needs.

Install suggested plugins

Install plugins the Jenkins community finds most useful.

Select plugins to install

Select and install plugins most suitable for your needs.

Nous allons cliquer sur l'option **Installer les plugins suggérés**, qui lancera immédiatement le processus d'installation.

Getting Started

✓ Folders	✓ OWASP Markup Formatter	✓ Build Timeout	✓ Credentials Binding	** Pipeline: Milestone Step
✓ Timestampers	✓ Workspace Cleanup	✓ Ant	✓ Gradle	** JavaScript GUI Lib: jQuery bundles (jQuery and jQuery UI)
🔗 Pipeline	🔗 GitHub Branch Source	🔗 Pipeline: GitHub Groovy Libraries	✓ Pipeline: Stage View	** Jackson 2 API
🔗 Git	🔗 Subversion	🔗 SSH Slaves	🔗 Matrix Authorization Strategy	** JavaScript GUI Lib: ACE Editor bundle
🔗 PAM Authentication	🔗 LDAP	🔗 Email Extension	🔗 Mailer	** Pipeline: SCM Step
				** Pipeline: Groovy
				** Pipeline: Input Step
				** Pipeline: Stage Step
				** Pipeline: Job
				** Pipeline: Graph Analysis
				** Pipeline: REST API
				** JavaScript GUI Lib: Handlebars bundle
				** JavaScript GUI Lib: Moment.js bundle
				Pipeline: Stage View
				** Pipeline: Build Step
				** Pipeline: Model API
				** Pipeline: Declarative Extension Points API
				** Apache HttpComponents Client 4.x API
				** JSch dependency

Une fois l'installation terminée, vous serez invité à configurer le premier utilisateur administratif. Vous pouvez sauter cette étape et continuer comme admin en utilisant le mot de passe initial utilisé ci-dessus, mais nous allons prendre quelques minutes pour créer l'utilisateur.

Getting Started

Create First Admin User

Username:

Password:

Confirm password:

Full name:

E-mail address:

Jenkins 2.121.1

[Continue as admin](#)

Saisissez le nom et le mot de passe de votre utilisateur :

Vous verrez apparaître une page **Instance configuration** qui vous demandera de confirmer l'URL préférée de votre instance Jenkins. Vérifiez qu'il s'agit bien du nom de domaine ou de l'adresse IP de votre serveur :

Getting Started

Instance Configuration

Jenkins URL:

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the `BUILD_URL` environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

Jenkins 2.121.1

Not now

Save and Finish

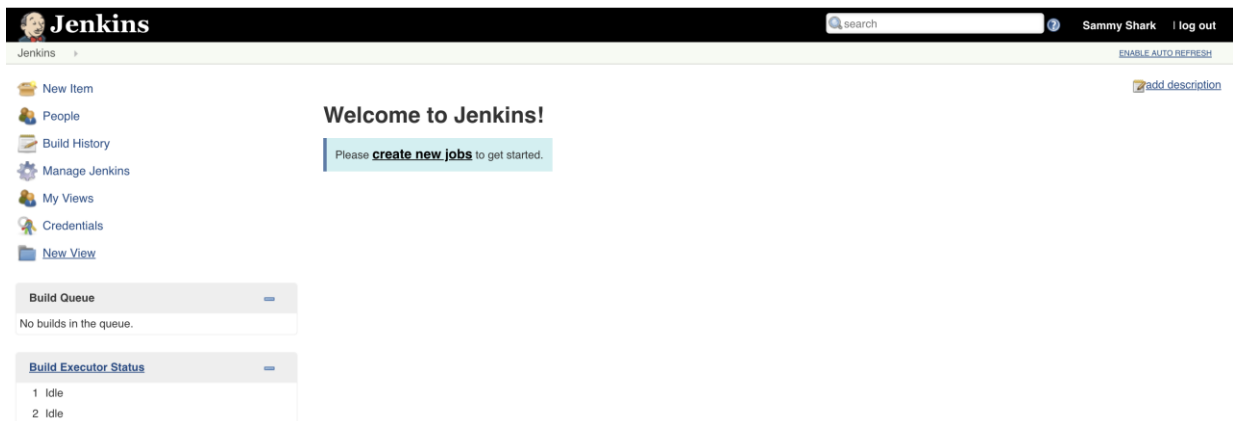
Après avoir confirmé l'exactitude des informations, cliquez sur **Save and Finish** Vous verrez apparaître une page de confirmation confirmant que « **Jenkins est prêt !** » :

Jenkins is ready!

Your Jenkins setup is complete.

Start using Jenkins

Cliquez sur **Start using Jenkins** pour consulter le tableau de bord principal de Jenkins :



The screenshot shows the Jenkins dashboard interface. At the top, there's a black header with the Jenkins logo, a search bar, and user information (Sammy Shark, log out). Below the header, the main content area is divided into a left sidebar with navigation links (New Item, People, Build History, Manage Jenkins, My Views, Credentials, New View) and a central area. The central area displays a 'Welcome to Jenkins!' message with a prompt to 'create new jobs'. Below this, there are two sections: 'Build Queue' showing 'No builds in the queue.' and 'Build Executor Status' showing two 'Idle' executors.

Jenkins

search

Sammy Shark | log out

ENABLE AUTO REFRESH

add description

New Item

People

Build History

Manage Jenkins

My Views

Credentials

New View

Build Queue

No builds in the queue.

Build Executor Status

1 Idle

2 Idle

À ce stade, vous avez terminé d'installer correctement Jenkins.

3. création d'un pipeline pour le Projet nodeapp

Lien sur gitlab: <https://gitlab.com/drissRiane/nodeapp.git>

git branch: 'main', url: 'https://gitlab.com/drissRiane/nodeapp.git'

- Cliquez sur 'nouveau item' choisissez le type pipeline et donnez un nom au pipeline par ex : nodepipeline

2. Script du pipeline

```
pipeline {
  environment {
    registry = "192.168.148.118:5000/nodeapp"
    dockerImage = ""
  }
  agent any
  stages {
    stage('Cloning our Git') {
      steps {
        git branch: 'main', url: 'https://gitlab.com/drissRiane/nodeapp.git'
      }
    }
    stage('Building our image') {
      steps{
```

```
script {  
  
  dockerImage = docker.build registry + ":$BUILD_NUMBER"  
  
}  
  
}  
  
stage('Deploy our image') {  
  
  steps{  
  
    script {  
  
      docker.withRegistry( "192.168.148.118:5000") {  
  
        dockerImage.push()  
  
      }  
  
    }  
  
  }  
  
}  
  
stage('Cleaning up') {  
  
  steps{  
  
    sh "docker rmi $registry:$BUILD_NUMBER"  
  
  }  
  
}  
  
}
```

Lancer le build

Deploy to kubernetes minikube

- Jenkins Plugins à installer:

- Docker Version 1.2.9 — This plugin integrates Jenkins with Docker
- Docker Pipeline Version 1.29 — Build and use Docker containers from pipelines.
- Kubernetes plugin — This plugin integrates Jenkins with Kubernetes

Créer un service account avec secret pour kubernetes-plugin in minikube

Creer un Fichier account.yaml

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins
  namespace: default
---

kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: jenkins
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods", "services"]
  verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]
- apiGroups: [""]
  resources: ["pods/exec"]
```

```

    verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]
- apiGroups: [""]
  resources: ["pods/log"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["secrets"]
  verbs: ["get"]
- apiGroups: [""]
  resources: ["persistentvolumeclaims"]
  verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]

---
apiVersion: v1
kind: Secret
metadata:
  name: jenkins-token
  annotations:
    kubernetes.io/service-account.name: jenkins
type: kubernetes.io/service-account-token

---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: jenkins
  namespace: default
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: jenkins
subjects:
- kind: ServiceAccount
  name: jenkins

---
# Allows jenkins to create persistent volumes
# This cluster role binding allows anyone in the "manager" group to
read secrets in any namespace.
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: jenkins-crb
subjects:
- kind: ServiceAccount
  namespace: default
  name: jenkins
roleRef:
  kind: ClusterRole
  name: jenkinsclusterrole
  apiGroup: rbac.authorization.k8s.io

---
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  # "namespace" omitted since ClusterRoles are not namespaced

```

```
name: jenkinsclusterrole
rules:
- apiGroups: [""]
  resources: ["persistentvolumes"]
  verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]
```

➤ `kubectl apply -f account.yaml`

Save secret token data, we need it while configure Kubernetes credentials in Jenkins

```
kubectl get secrets
```

```
kubectl describe secrets/jenkins-token
```

In Jenkins click on Manage Jenkins → Manage Credentials

Secret: token string , Paste token data in a secret field

- **create Kubernetes Id (my_kubernetes) with token data(token string)**
- **Save DockerHub credentials in a Jenkins**
- **Configuring Kubernetes plugin**

Jenkins — manage Jenkins — Manage Nodes & Clouds

Configure system scroll to bottom and in Add a new cloud, select Kubernetes

Convert certificate info into base64 encoding and paste it in a field (**Kubernetes server certificate key**)

```
cat /home/driss/.minikube/ca.crt | base64 -w 0; echo
```

Use Kubernetes Id(my_kubernetes) which was crated earlier.

```
nodejsapp.yaml
---
kind: Deployment
apiVersion: apps/v1
metadata:
  name: nodejs-app
  namespace: default
  labels:
    app: nodejs-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nodejs-app
  template:
    metadata:
      labels:
        app: nodejs-app
    spec:
      containers:
        - name: nodejs-app
          image: "192.168.148.118:5000/nodeapp"
          ports:
            - containerPort: 3000
---
apiVersion: v1
```

```
kind: Service
metadata:
  name: nodejs-app
  namespace: default
spec:
  selector:
    app: nodejs-app
  type: nodePort
  ports:
    - name: http
      targetPort: 3000
      port: 80
```

Ajouter stage deploy to minikube dans votre pipeline

```
//Ajouter ce stage à votre pipeline

stage('Deploy to minikube') {
  steps {
    withCredentials([
      string(credentialsId: 'my_kubernetes', variable:
'api_token')
    ]) {
      sh 'kubectl --token $api_token --server
https://192.168.49.2:8443 --insecure-skip-tls-verify=true apply -f
nodejsapp.yaml '
    }
  }
}
```

Test application